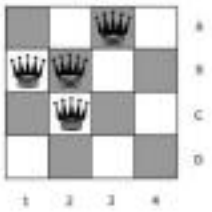
BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Phạm Tân Đạt
MSSV / Lớp	3123410072

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Thuật toán Hill-climbing thực hiện việc di chuyển một quân hậu trong cột của nó sao cho trạng thái kế tiếp (neighbor) có số lượng xung đột giảm nhiều nhất (hoặc thấp nhất).

Từ trạng thái ban đầu, ta có thể di chuyển quân hậu ở cột 1 từ hàng 2 lên hàng 1 (hoặc dịch chuyển quân hậu cột 2 lên hàng 2), số xung đột giảm từ 1 xuống 0.

Trạng thái sau một vòng lặp tối ưu: Cột 1: hàng 1, Cột 2: hàng 3, Cột 3: hàng 4, Cột 4: hàng 2 (hoặc một cấu hình đạt nghiệm 0 xung đột tùy theo lựa chọn dịch chuyển bước đầu). Cụ thể, đưa hậu cột 1 về hàng 1 sẽ triệt tiêu hoàn toàn xung đột chéo với cột 2. Trạng thái sau một vòng lặp:

- Quân hậu cột 1: Hàng 1
- Quân hậu cột 2: Hàng 3
- Quân hậu cột 3: Hàng 4
- Quân hậu cột 4: Hàng 2

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

☐ B) Cực trị cục bộ (local optimum)

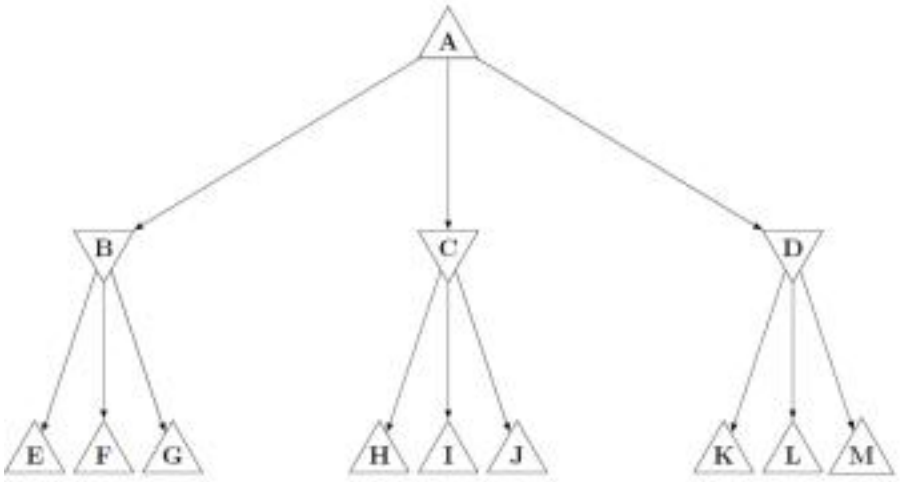
☑ C) Cực trị toàn cục (global optimum)

☐ D) Vai bằng (shoulder)

☐ E) Không thuộc các trường hợp trên

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ $B \rightarrow C \rightarrow D$

☐ $C \rightarrow D \rightarrow B$

☑ $D \rightarrow C \rightarrow B$

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Sau khi duyệt qua cây con D (mang lại giá trị ≥ 19 cho MAX), khi chuyển sang duyệt cây con C (là nút MIN): nút C sẽ tìm giá trị nhỏ hơn hoặc bằng các nút lá của nó (H, I, J). Nếu nhánh đầu tiên của C đã nhỏ hơn alpha hiện tại, các nhánh sau của C và toàn bộ cây B sẽ bị cắt tỉa.

Liệt kê các nút bị cắt tỉa: Tùy thuộc vào giá trị cụ thể của các nút lá H, I, J và E, F, G, khi áp dụng cắt tỉa Alpha-Beta với thứ tự $D \rightarrow C \rightarrow B$, cây B (gồm các nút B, E, F, G) sẽ bị cắt tỉa hoàn toàn do giá trị của D đã quá lớn áp đảo nhánh bên phải.

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

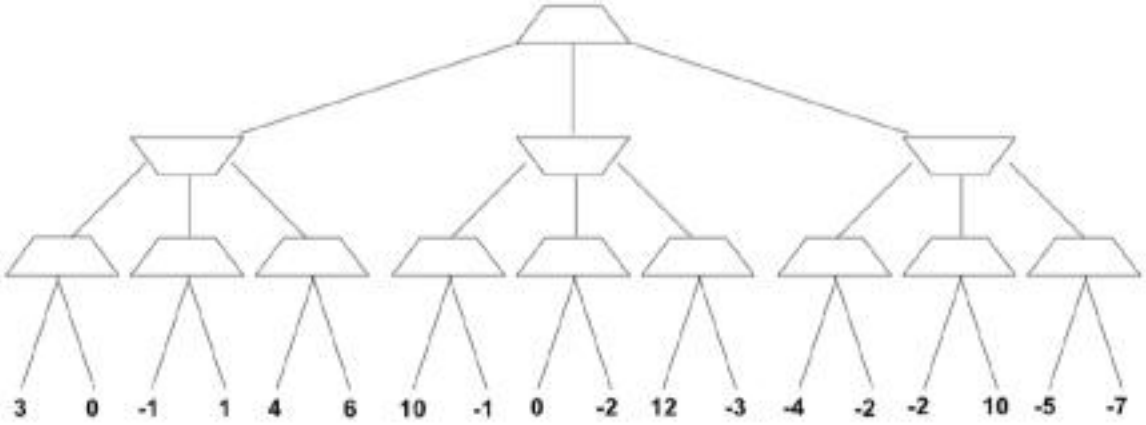
☐ Có

☑ Không

Giải thích: Việc giới hạn độ sâu (depth-limited) trong iterative deepening làm thay đổi hàm đánh giá tại các lớp trung gian (dùng heuristic thay vì giá trị đích thực tế của trò chơi), dẫn đến việc thứ tự sắp xếp các cây con có thể không phản ánh đúng giá trị tối ưu thực sự của toàn bộ cây, gây ra sai lệch về nước đi tối ưu so với Minimax duyệt đầy đủ.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

- Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.
- Ghi giá trị được trả về tại mỗi nút trung gian.
- Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.
- Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Quá trình duyệt Alpha-Beta (với gốc MAX, khởi tạo $\alpha = -\infty, \beta = +\infty$):

1. Duyệt nhánh MIN thứ nhất:
   - Các lá: $3,0 \rightarrow$ MIN chọn $min(3,0) = 0$.
   - Các lá: $-1,1 \rightarrow min(-1,1) = -1$.
   - Các lá: $4,6 \rightarrow min(4,6) = 4$.
   - Nút MIN thứ nhất trả về giá trị lớn nhất trong các giá trị con của nó $max(0, -1,4) = 4$. Cập nhật $\alpha = 4$.
2. Duyệt nhánh MIN thứ hai:
   - Các lá: $10, -1 \rightarrow min = -1$.
   - Các lá: $0, -2 \rightarrow min = -2$.
   - Các lá: $12, -3 \rightarrow min = -3$.
   - Nút MIN thứ hai trả về $max(-1, -2, -3) = -1$.
3. Duyệt nhánh MIN thứ ba:
   - Các lá: $-4, -2 \rightarrow min = -4$.
   - Các lá: $-2,10 \rightarrow min = -2$.
   - Các lá: $-5, -7 \rightarrow min = -7$.
   - Nút MIN thứ ba trả về $max(-4, -2, -7) = -2$.

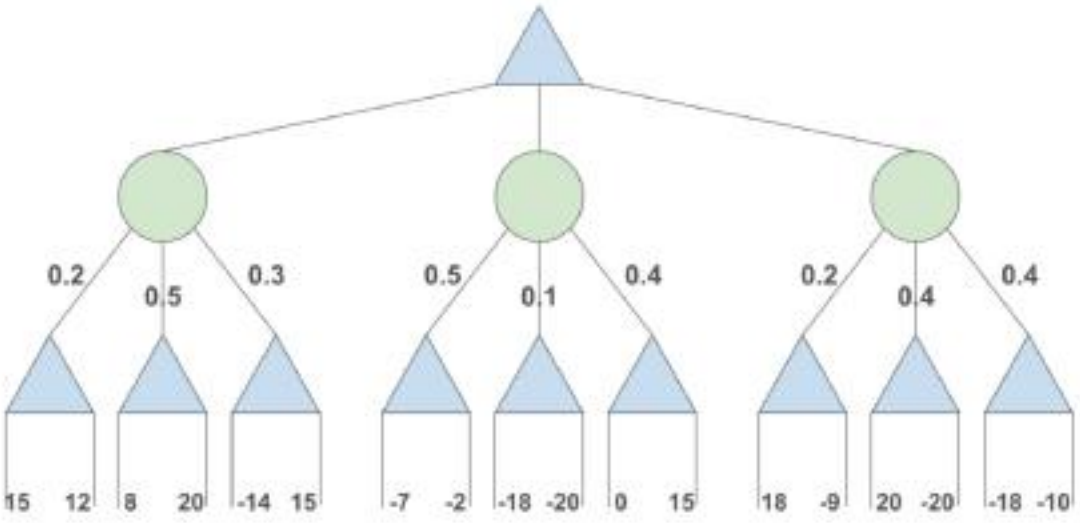
Kết quả:

- Giá trị Minimax tại nút gốc: $max(4, -1, -2) = 4$.
- Nước đi tối ưu của MAX: Chọn nhánh con đầu tiên (dẫn đến giá trị 4).

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác

suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Nút MAX tầng dưới (tính giá trị max của các lá từ trái sang phải):

- Nhóm 1: $max(15,12) = 15$; $max(8,20) = 20$; $max(-14,15) = 15$.
- Nhóm 2: $max(-7,-2) = -2$; $max(-18,-20) = -18$; $max(0,15) = 15$.
- Nhóm 3: $max(18,-9) = 18$; $max(20,-20) = 20$; $max(-18,-10) = -10$.

Chance nodes (tính kỳ vọng - trung bình có trọng số theo xác suất):

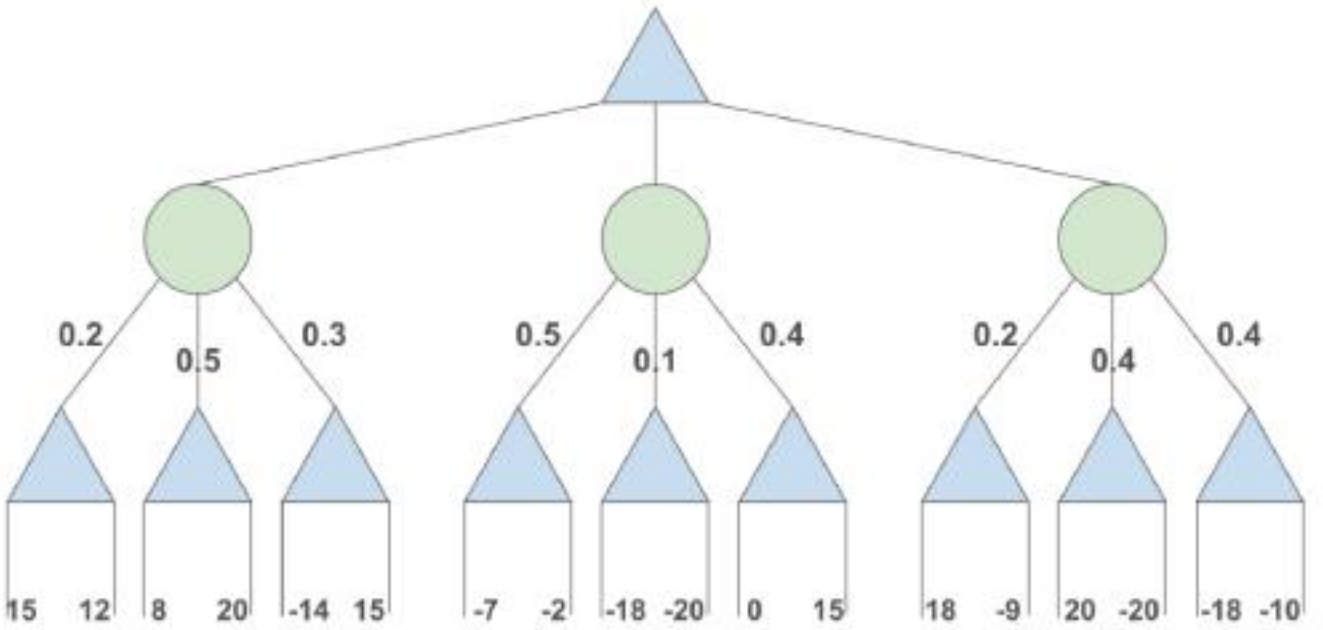
- Chance 1: $(15 \times 0.2) + (20 \times 0.5) + (15 \times 0.3) = 3 + 10 + 4.5 = 17.5$.
- Chance 2: $(-2 \times 0.5) + (-18 \times 0.1) + (15 \times 0.4) = -1.0 - 1.8 + 6.0 = 3.2$.
- Chance 3: $(18 \times 0.2) + (20 \times 0.4) + (-10 \times 0.4) = 3.6 + 8.0 - 4.0 = 7.6$.

Nút gốc MAX:$max(17.5,3.2,7.6) = 17.5$.

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Nếu không biết giới hạn giá trị lá, không thể cắt tỉa được nút nào trong cây Expectimax. Lý do là các nút chance tính giá trị kỳ vọng (trung bình cộng), nên một giá trị lá cực lớn hoặc cực nhỏ ở các nhánh sau vẫn có thể làm thay đổi kết quả kỳ vọng tổng thể.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Ngay cả khi biết giới hạn, việc cắt tỉa trong cây Expectimax rất hạn chế trừ khi tổng xác suất còn lại không đủ để vượt qua giá trị tốt nhất hiện tại (Alpha-Beta pruning thông thường không áp dụng trực tiếp cho chance node trừ khi có chặn trên/chặn dưới xác suất cụ thể). Với các giá trị trong đoạn $[-20,20]$, ta tính toán tương tự câu (a) vì các giá trị đều nằm trong biên độ cho phép.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Phạm vi lan truyền ràng buộc:

- Forward Checking (FC): Chỉ kiểm tra và loại bỏ các giá trị không tương thích ở các biến chưa gán có chung ràng buộc nhị phân trực tiếp với biến vừa được gán.
- AC-3 (Arc Consistency 3): Lan truyền tính nhất quán trên toàn bộ đồ thị ràng buộc (lan truyền rộng hơn, khi một biến bị hẹp miền thì các cung liên quan đến nó tiếp tục được kiểm tra lại).

Thời điểm thực hiện:

- FC: Thực hiện ngay sau mỗi bước gán giá trị cho một biến.
- AC-3: Thường chạy như một bước tiền xử lý trước khi tìm kiếm (preprocessing), hoặc được gọi lặp lại trong quá trình tìm kiếm.

Chi phí tính toán: FC có chi phí thấp hơn nhưng khả năng phát hiện deadlock chậm hơn AC-3; AC-3 có chi phí tính toán lớn hơn (quản lý hàng đợi các cung) nhưng loại bỏ được nhiều giá trị xung đột sớm hơn.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV): Chọn biến có ít giá trị hợp lệ còn lại nhất trong miền (biến "thắt cổ chai" / dễ

fail nhất) để gán trước. Giúp phát hiện sớm các thất bại để nhánh cây tìm kiếm không bị phình to vô ích.

ii) Least Constraining Value (LCV): Chọn giá trị cho biến sao cho loại bỏ ít giá trị nhất trong miền của các biến chưa gán có liên quan. Giúp tối ưu hóa cơ hội tìm thấy nghiệm ở các bước tiếp theo (tăng khả năng thành công cho các bước sau).

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

- Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.
- Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.
- Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.
- Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng. a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

Biến: A, B, C, D, E (mỗi máy bay tương ứng với một biến đại diện cho khung giờ hạ cánh/cất cánh trên đường băng của nó).
Miền giá trị: $D_A = D_B = D_C = D_D = D_E = \{1,2,3,4\}$.
Ràng buộc Unary (đơn):

- $B = 1$
- $D \geq 3$ (tức là $D \in \{3,4\}$)
- $A \leq 2$ (tức là $A \in \{1,2\}$)

Ràng buộc Binary (nhị phân):

- Ràng buộc thứ tự: $D < C$ (vì D hạ cánh trước khi C cất cánh).
- Ràng buộc khác nhau trên cùng đường băng:
  - Đường băng quốc tế (A, B, C): $A \neq B, A \neq C, B \neq C$.
  - Đường băng nội địa (D, E): $D \neq E$.

Bổ sung hai ràng buộc sau:

- A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.
- D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Do đã phân tách đường băng (Quốc tế: A, B, C; Nội địa: D, E), các máy bay khác đường băng không xung đột trực tiếp với nhau về lịch trình đường băng.
Các cạnh trong đồ thị ràng buộc nhị phân: $(A, B), (A, C), (B, C), (D, E)$ và (D, C).

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc

unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

Unary trước:

- $B = \{1\}$
- $A = \{1,2\}$
- $D = \{3,4\}$
- $C = \{1,2,3,4\}$
- $E = \{1,2,3,4\}$

Lan truyền (AC):

- Vì A, B, C cùng đường băng và $B = 1$, suy ra A và C không thể bằng 1 → $A \in \{2\}$, $C \in \{2,3,4\}$.
- Vì $A = 2$, miền của A được gọt còn $\{2\}$.
- Vì $D < C$ và $D \in \{3,4\}$, suy ra C phải lớn hơn hoặc bằng 4 → $C = 4$, và $D = 3$ (hoặc 4, nhưng nếu $D = 4$ thì C không có giá trị nào lớn hơn, do đó $D = 3$ và $C = 4$).
- E không bị ràng buộc thứ tự với ai ngoài việc khác D, với $D = 3 \rightarrow E \in \{1,2,4\}$.

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

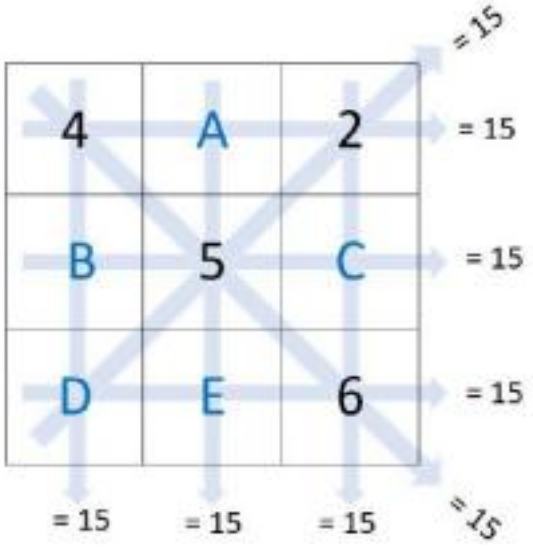
A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

1. Áp dụng Unary: $B = 1$ (gán cứng do đề bài).
2. Xét các biến còn lại (A, C, D, E):
    - Chọn biến theo MRV: A có miền $\{1,2\}$, sau khi $B = 1$, miền A còn $\{2\}$ (SL = 1). Chọn $A = 2$.
    - Tiếp theo xét D và C: $D \in \{3,4\}$, $C \in \{2,3,4\}$ (loại 2 do $A = 2$). Miền $C = \{3,4\}$, miền $D = \{3,4\}$.
    - Gán $D = 3$. Do ràng buộc $D < C$, miền của C cập nhật chỉ còn $\{4\}$. Gán $C = 4$.
    - Biến cuối cùng $E \in \{1,2,4\}$ (khác $D = 3 \rightarrow E \in \{1,2,4\}$). Chọn giá trị nhỏ nhất: $E = 1$ (nhưng xung đột với $B = 1$ nếu chung đường băng nội địa. E dùng đường băng nội địa nên không xung đột với B quốc tế). Do đó $E = 1$ hợp lệ.
- Thứ tự gán (biến, giá trị): $(B, 1) \rightarrow (A, 2) \rightarrow (D, 3) \rightarrow (C, 4) \rightarrow (E, 1)$.

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu

chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.
Biến: Các ô chưa biết ký hiệu là A, B, C, D, E (và các ô trống còn lại nếu có).
Miền giá trị: $\{1,2,3,4,6,7,8,9\}$ (các số từ 1 đến 9 trừ các số đã cho sẵn là $2,4,5,6$).
Ràng buộc:

- $AllDiff(A, B, C, D, E, \dots)$ (mỗi ô một giá trị phân biệt).
- Tổng các hàng, cột và 2 đường chéo đều bằng 15.
    - Hàng 1: $4 + A + 2 = 15 \rightarrow A = 9$.
    - Hàng 2: $B + 5 + C = 15 \rightarrow B + C = 10$.
    - Hàng 3: $D + E + 6 = 15 \rightarrow D + E = 9$.
    - Cột 1: $4 + B + D = 15 \rightarrow B + D = 11$.
    - Cột 3: $2 + C + 6 = 15 \rightarrow C = 7$

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.
Các biến A, B, C, D, E có các cạnh nối với nhau thông qua các ràng buộc tổng hàng/cột/đường chéo và ràng buộc khác nhau ($alldiff$).

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.
Từ phương trình cột 3: $2 + C + 6 = 15 \rightarrow C = 7$.
Từ hàng 1: $4 + A + 2 = 15 \rightarrow A = 9$.
Từ hàng 2 ($B + 5 + C = 15$ với $C = 7$): $B + 5 + 7 = 15 \rightarrow B + 12 = 15 \rightarrow B = 3$.
Từ cột 1 ($4 + B + D = 15$ với $B = 3$): $4 + 3 + D = 15 \rightarrow 7 + D = 15 \rightarrow D = 8$.
Từ hàng 3 ($D + E + 6 = 15$ với $D = 8$): $8 + E + 6 = 15 \rightarrow 14 + E = 15 \rightarrow E = 1$.
Kiểm tra các đường chéo:

- Đường chéo chính: $4 + 5 + 6 = 15$ (Đúng).
- Đường chéo phụ: $2 + 5 + D = 2 + 5 + 8 = 15$ (Đúng).

Nghiệm hoàn chỉnh của Magic Square:

- Hàng 1: $4,9,2$
- Hàng 2: $3,5,7$

- Hàng 3: *8,1,6*

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.
Chọn biến (MRV): Biến có ít giá trị nhất là $Ellyse$ (2 giá trị: Đỏ, Hồng). Chọn $Ellyse$.
Chọn giá trị cho $Ellyse$ (LCV):

- Nếu chọn *Đỏ*: loại Đỏ khỏi các tập màu khác (AJ và Sunny mất Đỏ).
- Nếu chọn *Hồng*: loại Hồng khỏi Sunny (AJ không có Hồng nên không ảnh hưởng).
- Tính số lượng giá trị bị loại (Least Constraining Value):
    - Chọn *Đỏ*: loại Đỏ ở AJ và Sunny (2 giá trị bị loại).
    - Chọn *Hồng*: loại Hồng ở Sunny (1 giá trị bị loại).

=> LCV chọn $Ellyse = Hồng$.

Tiếp theo (MRV): Xét AJ và $Sunny$ (còn lại 2 biến, mỗi biến có 2 màu khả dụng sau khi Sunny mất Hồng → $Sunny = \{Đỏ, Vàng\}$, $AJ = \{Đỏ, Vàng, Xanh\ lá\}$ nhưng đã mất Hồng thì không ảnh hưởng, AJ còn $\{Đỏ, Vàng, Xanh\ lá\}$ vì $Ellyse$ chọn Hồng).

- Chọn biến có ít giá trị nhất tiếp theo: $Sunny$ (còn 2 màu: Đỏ, Vàng).
- Chọn giá trị cho $Sunny$ (LCV):
    - Nếu chọn *Đỏ*: loại Đỏ ở AJ.
    - Nếu chọn *Vàng*: loại Vàng ở AJ.
    - Cả hai đều loại 1 giá trị ở AJ. Phá hòa theo thứ tự cầu vồng chọn màu Đỏ → $Sunny = Đỏ$.

Biến cuối cùng AJ:

    - Vì $Ellyse = Hồng$ và $Sunny = Đỏ$, tập của AJ còn lại: Xanh lá (vì Đỏ và Hồng đã bị lấy).
    - Gán $AJ = Xanhlá$.
- Số lần backtrack: 0 lần (thuật toán tìm thấy ngay nghiệm hợp lệ không gặp dead end).

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

Maximum Remaining Values: Chọn biến có nhiều giá trị nhất trước → Chọn AJ (3 giá trị).
Most Constraining Value: Chọn giá trị loại bỏ nhiều nhất các giá trị của biến liên quan.
Việc đảo ngược heuristic này thường làm tăng không gian tìm kiếm, dễ dẫn đến các lựa chọn xung đột ở các bước sau, làm tăng số lần phải backtrack (thường từ 1 đến 2 lần backtrack tùy thuộc vào nhánh xung đột màu Đỏ/Vàng).